

ICS2121 Métodos de Optimización

Algunos elementos “avanzados” de Programación Entera

Cómo abordamos problemas de Programación Entera

- Hemos estado preocupados de abordar problemas estructurados y muy grandes.
- Muchos de esas problemas son de Programación Entera (o enteros mixtos).
- También aparecen problemas enteros en las etapas de los métodos de descomposición: en el maestro y/o el satélite.
- Vamos a repasar y reforzar lo que sabemos de técnicas para problemas con variables enteras.

Cómo abordamos problemas de Programación Entera

- Consideremos un problema entero de la siguiente forma:

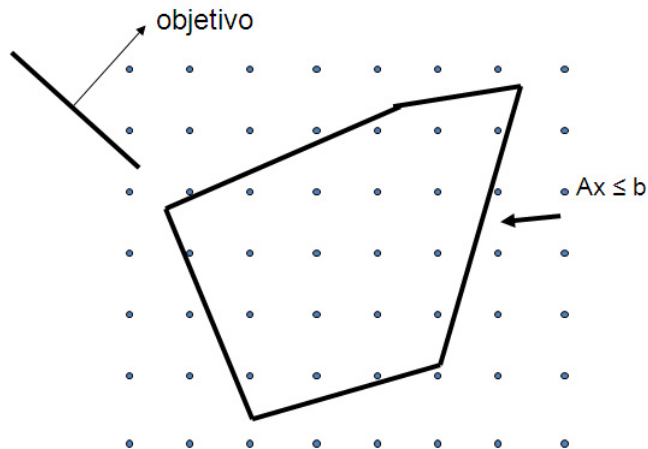
$$\begin{array}{ll} IP) & z^* = \min \quad c^T x \\ & s.a. \quad Ax \leq b \\ & \quad \quad x \geq 0, \text{ entero} \end{array}$$

(también puede estar en forma estándar)

- Supongamos inicialmente que la región factible es acotada.

¿Cómo podemos resolver un problema entero?

- Gráficamente:



¿Cómo podemos resolver un problema entero?

- Si bien hay un número finito de posibles soluciones, es exponencialmente enorme...
- Pero hay una manera “inteligente” de buscar una solución óptima.
- Supongamos que resolvemos la relajación lineal:

$$P_0) \quad \begin{array}{ll} z^0 = \text{mín} & c^T x \\ & s.a. \quad Ax \leq b \\ & \quad \quad x \geq 0 \end{array}$$

- Sea x^0 solución óptima de la relajación lineal.
- ¿Qué pasa si todas las componentes de x^0 son enteras?
- Si no, sea k un índice tal que x_k^0 sea fraccionaria.

¿Cómo podemos resolver un problema entero?

- Ahora creamos dos problemas a partir de la relajación lineal:

$$P_1) \quad \begin{array}{ll} z^1 = \min & c^T x \\ \text{s.a.} & Ax \leq b \\ & x \geq 0 \\ & \boxed{x_k \leq [x_k^0]} \end{array}$$

$$P_2) \quad \begin{array}{ll} z^2 = \min & c^T x \\ \text{s.a.} & Ax \leq b \\ & x \geq 0 \\ & \boxed{x_k \geq [x_k^0] + 1} \end{array}$$

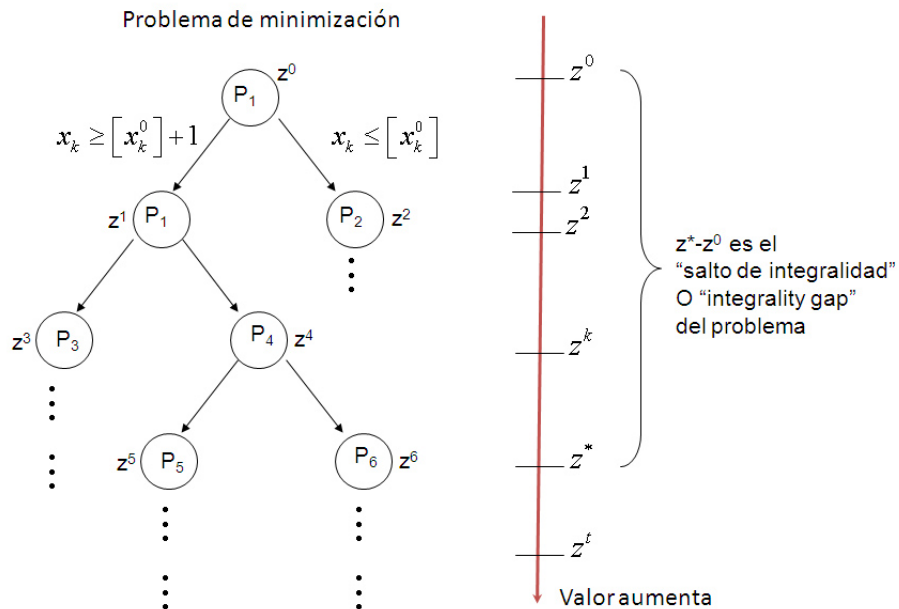
(donde $[u]$ denota la parte entera de u .)

- Qué relación hay entre z^0, z^1, z^2 ?
- Es x^0 factible para alguno de los problemas $P_1)$ o $P_2)$?
- Esta idea es la base del Algoritmo de Ramificación y Acotamiento...

El algoritmo de Ramificación y Acotamiento

- Se desarrolla entonces un “árbol binario” en que cada nodo es una división del anterior.
- En algún momento algunas cosas pueden pasar en un nodo:
 - ▶ **Podemos encontrar un problema infactible**
 - ▶ en este caso la rama es “podada”
 - ▶ **Encontramos una solución entera**
 - ▶ Es esta una solución óptima?
- Sea w el valor de la mejor solución entera encontrada hasta el momento (este valor se llama “incumbente”).
- Supongamos que en un nodo encontramos una solución fraccionaria con valor z' .
- Qué podemos decir si $z' \geq w$?
- De este modo el árbol de ramificación se desarrolla con algunas ramas que se podan, ya sea por infactibilidad o por acotamiento.
- De ahí el nombre del procedimiento, que en Inglés se llama Branch and Bound.

El algoritmo de Ramificación y Acotamiento



El algoritmo de Ramificación y Acotamiento

- Esto es el Algoritmo de Ramificación y Acotamiento, o Branch and Bound.
- Fue desarrollado por Ailsa Land y Alison Doig en 1960, en el London School of Economics.



(imágenes de wikipedia.com, dominio público)

- Muchas veces los problemas son “binarios”:
- En ese caso, la ramificación fija valores en 1 o 0.
- Pero la lógica general es la misma...

Ramificación y Acotamiento: algunos comentarios y preguntas

- Cada subproblema “hijo” se diferencia del “padre” en que tiene una restricción adicional (o una variable fijada).
- La solución del “padre” puede usarse como punto de partida para el Simplex en el “hijo”, y eso se resuelve mediante Simplex, en versión dual.
- **¿En que orden generamos el árbol?:**
- En una etapa cualquiera habrá una lista de nodos pendientes de ser revisados...
- Qué problema seleccionamos para ramificar?
 - ▶ “depth first” (en profundidad)
 - ▶ “breath first” (hacia lo ancho)

Ramificación y Acotamiento: algunos comentarios y preguntas

- En la solución de un nodo particular puede haber más de una variable fraccionaria.
- **Qué variables seleccionamos para ramificar?**
- Debiera ser la que aporte más a “integralizar” la solución.
- Esto lleva a considerar distintos tipos de “prioridades” para las variables...
- Se puede priorizar por costo, por ejemplo.
- Pero hay otros criterios.
- “Strong Branching” es uno que se implementa frecuentemente:
- “Probar” ramificaciones y estimar cuánto cambiará la función objetivo, realizando algunas iteraciones de Simplex.

Ramificación y Acotamiento: algunas preguntas

- **Cuándo nos detenemos?**

- Idealmente en el óptimo, pero eso puede tardar mucho tiempo...
- La idea es evaluar el “salto (gap) de integralidad” entre la mejor solución obtenida hasta el momento y la mejor cota relajada disponible.
- Si tenemos una solución factible (no necesariamente óptima), podemos usarla como incumbente inicial.
- ¿Y qué pasa si el problema es infactible...?

Ramificación y Acotamiento: ejemplo

- Veamos un ejemplo “real”.
- Para esto consideremos nuevamente el problema, de localización y asignación de clientes que presentamos en el tema de Relajación Lagrangiana:

$$\begin{aligned} \text{mín} \quad & \sum_{i=1}^m f_i y_i + \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.a.} \quad & \sum_{i=1}^m x_{ij} = 1, & j = 1, \dots, n \end{aligned} \tag{1}$$

$$\begin{aligned} & \sum_{j=1}^n x_{ij} \leq n y_i, & i = 1, \dots, m \\ & y_i, x_{ij} \in \{0, 1\} & i = 1, \dots, m; j = 1, \dots, n \end{aligned} \tag{2}$$

- Vamos a ver en más detalle la resolución mediante Branch and Bound.

Más allá del Branch & and Bound

- Podemos ver que el algoritmo de B&B se comporta diferente según parámetros de Gurobi, pero también según el modelo...
- Es decir, dos formas, totalmente equivalentes, de escribir el problema dan ejecuciones diferentes.
- Tratemos de entender lo que ocurre aquí...

La importancia de la formulación “correcta”

- Consideremos, como antes, un problema entero de la forma

$$\begin{array}{ll} z^* = \min & c^T x \\ IP) & s.t. \quad Ax \leq b \\ & x \text{ entero} \end{array}$$

- Sea $S = \{x \in \mathbb{R}^n : Ax \leq b, x \text{ entero}\}$ el conjunto de puntos enteros factibles para IP).

Definición

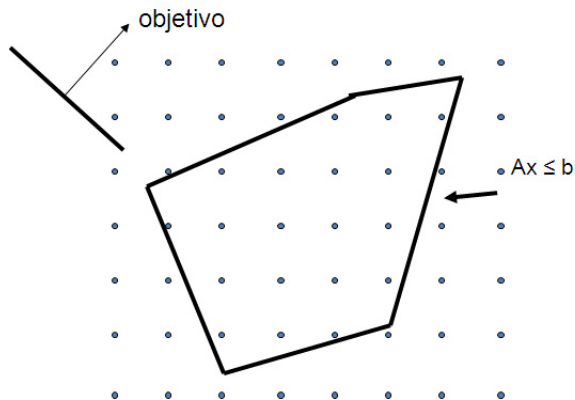
Un sistema lineal de desigualdades $Dx \leq d$ se dice que es una “**formulación**” para el problema entero IP) si

$$\{x \in \mathbb{R}^n : Dx \leq d, x \text{ entero}\} = S$$

- Es decir, es otra forma alternativa de describir el mismo problema.
- En el ejemplo, usamos dos formulaciones diferentes, pero el problema era el mismo...

La importancia de la formulación “correcta”

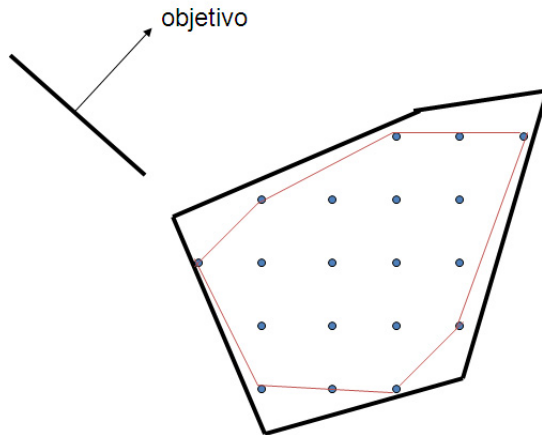
- ¿Qué está ocurriendo?
- Acá está, nuevamente, la geometría de las restricciones y puntos factibles.



- ¿Y si construyéramos la envoltura convexa de esos puntos enteros?

La importancia de la formulación “correcta”

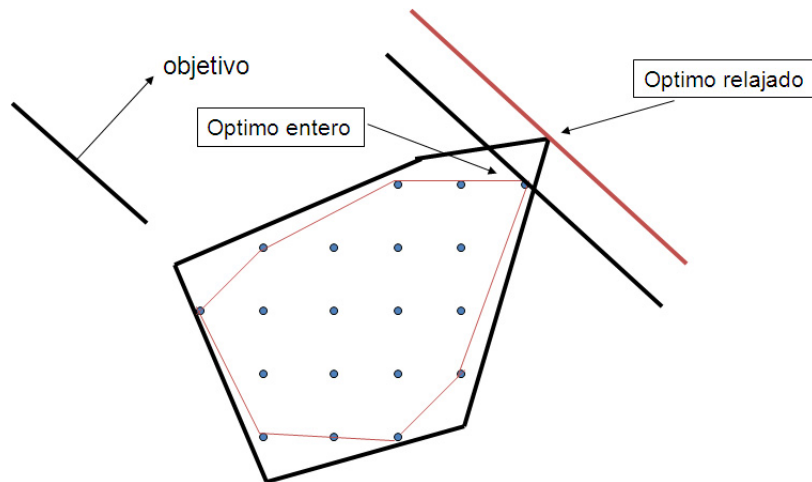
- denotemos por $CH(S)$ esa envoltura convexa.



- Por el Teorema de Weyl-Minkowsky-Motzkin, esa envoltura convexa es un poliedro!

La importancia de la formulación “correcta”

- Como $CH(S)$ es un poliedro, entonces existe una matriz D y vector d tales que $CH(S) = \{x : Dx \leq d\}$.
- Luego, podemos obtener la solución óptima de IP) resolviendo un problema de Programación Lineal sobre $CH(S)$.



La importancia de la formulación “correcta”

Definición

Al sistema $Dx \leq d$ tal que $CH(S) = \{x : Dx \leq d\}$ se le llama “**Formulación Ideal**” del problema IP).

- Un problema entero puede admitir muchas formulaciones, todas válidas.

Definición

Dadas dos formulaciones válidas para el problema IP):

F1: dada por el sistema $D^1x \leq d^1$

F2: dada por el sistema $D^2x \leq d^2$,

decimos que F1 “es más fuerte” que F2 si

$$\{x \in R^n : D^1x \leq d^1\} \subset \{x \in R^n : D^2x \leq d^2\}$$

La importancia de la formulación “correcta”

- Sean

$$z^1 = \min\{c^T x : D^1 x \leq d^1\}$$

y

$$z^2 = \min\{c^T x : D^2 x \leq d^2\}$$

- Sea z^* el valor óptimo del problema entero original.
- Entonces, si F1 es más fuerte que F2, se tiene que

$$z^* \geq z^1 \geq z^2.$$

- La mejora en el valor de la relajación lineal es un indicador de cómo se fortalece la formulación, tal como ocurría en el ejemplo del Branch and Bound.

La importancia de la formulación “correcta”

- El argumento anterior muestra que si conociéramos la Formulación Ideal, podríamos resolver IP) como un “simple” problema de Programación Lineal!
- Pero: encontrar la formulación ideal es muy difícil o casi imposible.
- O, aún si la conociéramos, resolverla como Programación Lineal podría resultar muy difícil (podría tener un número exponencial de restricciones).
- Pero podemos aproximarnos....
- **Idea:** usar una formulación “fuerte” de modo que aproxime mejor a $CH(S)$, se “acerque” a la formulación ideal.
- Los cambios de formulación en el ejemplo, hacen precisamente eso.

Algunos problemas enteros resultan tener la formulación ideal a la vista

- Consideremos el siguiente problema, conocido como problema de asignación (y que ustedes deben conocer).

$$\begin{aligned} \text{mín} \quad & \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.a.} \quad & \sum_{j=1}^n x_{ij} = 1, i = 1 \dots, n \\ & \sum_{i=1}^n x_{ij} = 1, j = 1 \dots, n \\ & x_{ij} \in \{0, 1\}, i, j = 1, \dots, n \end{aligned}$$

Algunos problemas enteros resultan tener la formulación ideal a la vista

- Este problema tiene la propiedad de que la relajación lineal del problema entrega ya valores enteros.
- ¿Por qué ocurre esto?
- **Definición:** Sea A una matriz de $m \times n$. Se dice que A es **unimodular** si el valor de cualquier subdeterminante de A es 1, 0 ó -1 .

Teorema

Consideremos el problema lineal $\min\{c^T x : Ax = b, x \geq 0\}$. Supongamos que A es unimodular. Entonces, si b tiene componentes enteras, toda solución básica del problema tiene coordenadas enteras.

- (la demostración sigue de las propiedades de sistemas lineales...)
- En el problema de asignación, la matriz es unimodular.
- La unimodularidad es una propiedad muy importante, si se tiene...
- Desafortunadamente, la mayoría de los problemas enteros no tienen esta propiedad y son muy difíciles.

La Idea de Planos Cortantes en Programación Entera

- Consideremos un problema entero de la forma:

$$IP) \quad \begin{array}{ll} z^* = \text{mín} & c^T x \\ & \text{s.a. } Ax \leq b \\ & x \text{ entero} \end{array}$$

- Sea $S = \{x : Ax \leq b, \text{ entero}\}$ la región factible del problema.
- Ya hemos visto lo importante de fortalecer las formulaciones....
- Ahora veremos como hacerlo en forma más sistemática.
- **Definición:** $\alpha^T x \leq \beta$ se dice que es una **desigualdad válida** para S si

$$x \in S \implies \alpha^T x \leq \beta$$

La Idea de Planos Cortantes en Programación Entera

- Supongamos resolvemos la relajación lineal del problema:

$$P_0) \quad z^0 = \min_{s.a.} \quad c^T x \quad Ax \leq b.$$

- Sea x^0 solución óptima de la relajación lineal.
- Idea: construir una desigualdad **válida** $(\alpha^1)^T x \leq \beta^1$ tal que

$$(\alpha^1)^T x^0 > \beta^1$$

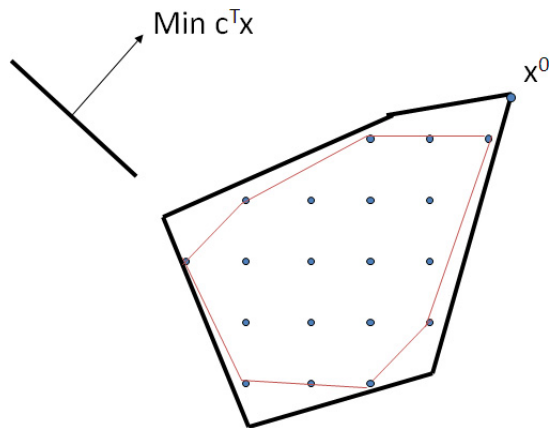
- esta nueva restricción “corta” la solución relajada.
- ahora resolvemos el problema:

$$P_1) \quad z^1 = \min_{s.a.} \quad c^T x \quad \begin{array}{l} Ax \leq b \\ (\alpha^1)^T x \leq \beta^1. \end{array}$$

- Y tenemos una nueva solución x^1 y valor z^1 tales que $z^* \geq z^1 \geq z^0$ (ojalá, $z^1 > z^0$).
- ¿Dónde hemos visto una idea parecida...?

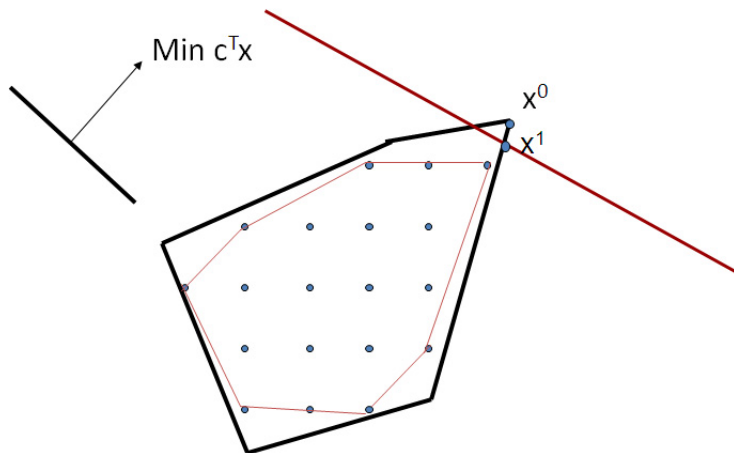
La Idea de Planos Cortantes en Programación Entera

- La idea es hacer esto sistemáticamente:



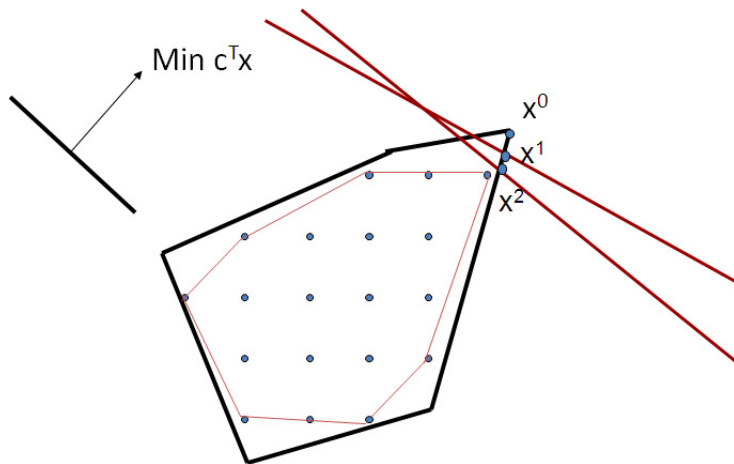
La Idea de Planos Cortantes en Programación Entera

- La idea es hacer esto sistemáticamente:



La Idea de Planos Cortantes en Programación Entera

- La idea es hacer esto sistemáticamente:



La Idea de Planos Cortantes en Programación Entera

- Un algoritmo general de planos cortantes:

0. Sea $S_0 = \{x : Ax \leq b\}$, $k = 0$.

1. Etapa general k :

- ▶ Resolver el problema lineal $\min\{c^T x : x \in S_k\}$.
- ▶ Sea x^k solución óptima.
- ▶ Si x^k es entero, PARAR, tenemos una solución óptima (o se satisface otro criterio de parada).

2. Si no, buscar una desigualdad válida $\alpha_k^T x \leq \beta_k$ tal que $\alpha_k^T x^k > \beta_k$.

3. $S_{k+1} = S_k \cap \{x : \alpha_k^T x \leq \beta_k\}$.

4. $k = k + 1$, ir a 1.

Planos cortantes: algunas preguntas

- Pregunta: ¿cómo generar planos cortantes?
- Un procedimiento que genera planos cortantes es lo que se conoce como un “oráculo de separación”.
- Por supuesto, el ideal sería poder generar las desigualdades de la formulación idea (“facetas”), pero en la mayoría de los problemas eso es muy difícil.
- Dos enfoques:
- **Procedimientos generales:**
- El primero y más famoso: el algoritmo de planos cortantes de Gomory.
 - ▶ Sirve para cualquier problema entero, garantiza encontrar una solución óptima después de un número finito (eventualmente muy grande) de cortes.
- Otras familias de cortes generales:
 - ▶ Covers, Flow Covers (están mencionados en el reporte de Gurobi)
 - ▶ Cortes basados en “convexificación”.
- **Cortes especializados que usan la estructura del problema.**

Primero tratemos con la aritmética

- Las propiedades de los número permiten generar desigualdades válidas.
- Consideremos, por ejemplo, la siguiente desigualdad:

$$3x_1 + 5x_2 \leq 10, x_i \in \{0, 1\}, i = 1, 2$$

- ¿Podemos fortalecerla?
- Ideas como estas ya están implementadas en Gurobi.
- Pero se pueden hacer cosas más sofisticadas.
- Una idea son los Cortes de Gomory (desarrollado por R. Gomory en IBM en 1958).
- Veremos en más detalle la idea de los “Covers”

Desigualdades de “Covers”

- Supongamos que un problema tiene la siguiente restricción:

$$3x_1 + x_2 + 5x_3 + 2x_4 \leq 6, \quad x_j \in \{0, 1\}, j = 1, \dots, 4$$

- Supongamos que se ha resuelto la relajación lineal del problema y se tiene la siguiente solución (relajada):

$$\bar{x}_1 = \frac{2}{3}, \bar{x}_2 = \frac{1}{2}, \bar{x}_3 = \frac{1}{2}, \bar{x}_4 = \frac{1}{2}$$

- Esta solución cumple la desigualdad anterior, pero no es entera...
- Sobre la base de los números, ¿hay alguna relación que siempre debería cumplirse para una solución entera?
- ¿Podemos decir cuántas variables x_j pueden ser > 0 ?

Desigualdades de “Covers”

- La original nuevamente:

$$3x_1 + x_2 + 5x_3 + 2x_4 \leq 6, \quad x_j \in \{0, 1\}, j = 1, \dots, 4$$

- Como $3 + 5 = 8 > 6$ se tiene que x_1 y x_3 NO pueden ser $= 1$ al mismo tiempo.
- Consideremos, entonces, la siguiente restricción:

$$x_1 + x_3 \leq 1 \quad (*)$$

- Esta restricción es completamente redundante con respecto a soluciones 0-1 de la restricción original.
- Ahora, notemos que

$$\bar{x}_1 + \bar{x}_3 = \frac{2}{3} + \frac{1}{2} = \frac{7}{6} > 1$$

- Es decir, la restricción $(*)$ “corta” la solución fraccionaria...

Desigualdades de “Covers”

- Consideremos una restricción “tipo knapsack” (mochila) binaria:

$$\sum_{j=1}^n a_j x_j \leq b, x_j \in \{0, 1\}, j = 1, \dots, n$$

- Observamos que siempre podemos asumir $a_j > 0, j = 1, \dots, n$ y $b > 0$.

Definición

$C \subset N$ se llama “cover” (o cubrimiento) si

$$\sum_{j \in C} a_j > b$$

Proposición

Si C es un cover, entonces la desigualdad

$$\sum_{j \in C} x_j \leq \text{card}(C) - 1$$

es válida para S .

Desigualdades de “Covers”

- Por ejemplo, busquemos covers en la siguiente desigualdad:

$$11x_1 + 6x_2 + 6x_3 + 5x_4 + 5x_5 + 4x_6 + x_7 \leq 19, x_j \in \{0, 1\}, j = 1, \dots, 7.$$

El problema de separación para “Covers”

- Dada una solución fraccionaria, ¿cómo podemos identificar un cover violado?
- Consideremos la restricción

$$\sum_{j=1}^n a_j x_j \leq b, \quad x_j \in \{0, 1\}, j = 1, \dots, n$$

- Supongamos que tenemos una solución fraccionaria $\bar{x}$.
- Primero observemos que la desigualdad de cover

$$\sum_{j \in C} x_j \leq \text{card}(C) - 1$$

se puede escribir

$$\sum_{j \in C} x_j < \text{card}(C) \Leftrightarrow \sum_{j \in C} x_j < \sum_{j \in C} 1 \Leftrightarrow \sum_{j \in C} (1 - x_j) \geq 1$$

El problema de separación para “Covers”

- Entonces, tenemos que encontrar $C \subset N$ tal que

$$\sum_{j \in C} (1 - \bar{x}_j) < 1$$

- Para esto plantearemos un problema de optimización entera.
- Sea $z_j \in \{0, 1\}$ tal que $z_j = 1$ si el índice j está en el cover, 0 si no.
- Entonces, formulamos el siguiente problema:

$$\begin{aligned} \bar{z} = \min \quad & \sum_{j=1}^n z_j (1 - \bar{x}_j) \\ \text{s.a.} \quad & \sum_{j=1}^n z_j a_j > b \\ & z_j \in \{0, 1\}, j = 1, \dots, n. \end{aligned}$$

- (Si b es entero la restricción se puede plantear como $\geq b - 1$.)
- Si $\bar{z} < 1$, hemos encontrado un cover que “corta” la solución actual.

El problema de separación para “Covers”

- De este modo, la separación de Covers requiere la solución de un problema de mochila.
- Estrictamente, el problema de mochila es NP-Completo, pero igual se puede resolver igual de manera eficiente...
- Recordemos además que no necesitamos la optimalidad: basta cumplir la condición de corte.
- Se han desarrollado ideas más sofisticadas que parten de los covers.
- Debe elegirse los de menor cardinalidad y, a partir de estos, es posible llegar a “facetas” de la formulación ideal del Problema de Mochila.

Cortes especializados a problemas

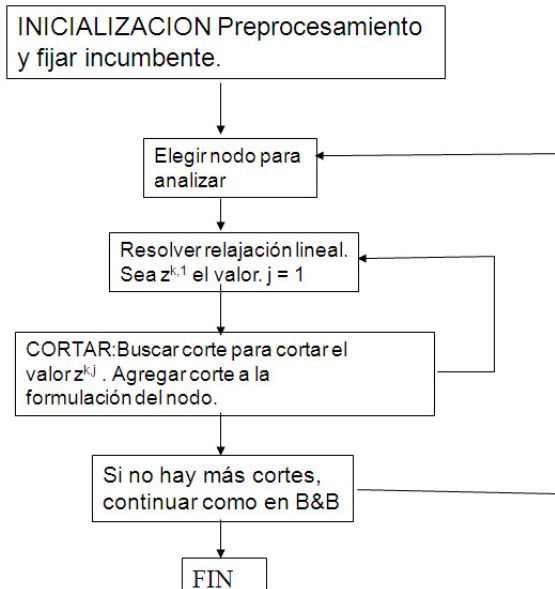
- Lo anterior eran procedimientos generales.
- Pero a veces la estructura del problema permite generar también desigualdades válidas muy buenas.
- Por ejemplo, consideremos un problema de diseño de redes y satisfacción de flujo:

Algoritmos de Branch and Cut

- Un algoritmo de planos cortantes agrega restricciones en forma iterativa...
- Finalmente o se alcanza una solución entera o nos detenemos y:
- Se usa el Branch and Bound sobre el problema con todos los cortes...
- Puede ser mejor idea ejecutar el Branch and Bound y simultaneamente ir agregando cortes...
- Esto es lo que realmente hace el Branch and Bound de Gurobi.

Algoritmos de Branch and Cut

- Esta es la idea general de un algoritmo de Branch and Cut:



- Los problemas enteros son difíciles, de hecho, muchas veces muy difíciles.
- Y no hay forma eficiente de abordarlos (a menos que $P = NP$).
- Pero existe mucho conocimiento sobre como abordar mejor los problemas.
- Todo esto es muy importante ya que, aún si usamos nuestras técnicas de descomposición, muchas veces igual terminamos con un problema entero difícil.